

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №7

З дисципліни БАЗИ ДАНИХ ТА ІНФОРМАЦІЙНІ СИСТЕМИ

Виконав
Студент 3 курсу
Групи МІТ-31
Факультету Інформаційних Технологій
Лаптев Олександр

Київ, 2026

Тема: Робота з Redis **Мета:** Закріпити розуміння роботи Redis та навчитися використовувати його основні можливості.

Вступ У даній лабораторній роботі досліджується In-Memory структура даних Redis. Розглядаються процеси встановлення сервера за допомогою контейнеризації, виконання базових команд для роботи з різними типами даних (рядки, списки, множини, хеші) та реалізація програмного підключення за допомогою мови Python

Хід роботи :

```
PS C:\Users\igarl> docker run --name redis-container -d -p 6379:6379 redis
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
1430dee9b11c: Pull complete
cb6adc4421fc: Pull complete
1c6351e0e0b6: Pull complete
bfe50995c2ff: Pull complete
924279091117: Pull complete
72c03230f136: Pull complete
4f4fb700ef54: Pull complete
ae14a01ff360: Download complete
18078f1a5219: Download complete
Digest: sha256:4c7ecf5d14207f5baf8e140ab428a54ae5cf2a0411956f75deb0ef89d3cb4fda
Status: Downloaded newer image for redis:latest
3f4e796a26bdab6e3becdcdd2df68f9c06b3d19e78e00a6a7d949310f47de5f0
PS C:\Users\igarl> docker exec -it redis-container redis-cli
127.0.0.1:6379> Ping
PONG
127.0.0.1:6379> |
```

Рис. 1. Підключення до Redis через Docker

```
127.0.0.1:6379> SET student "Oleksandr"
OK
127.0.0.1:6379> Get student
"Oleksandr"
127.0.0.1:6379> incr mycounter
(integer) 1
127.0.0.1:6379> incr mycounter
(integer) 2
127.0.0.1:6379> get mycounter
"2"
```

Рис. 2. Виконання базових операцій INCR , GET , SET

```
127.0.0.1:6379> lpush tasks "task1"
(integer) 1
127.0.0.1:6379> lpush tasks "task2"
(integer) 2
127.0.0.1:6379> lrange 0 -1
(error) ERR wrong number of arguments for 'lrange' command
127.0.0.1:6379> lrange tasks 0 -1
1) "task2"
2) "task1"
```

Рис. 3.Робота зі списками з використанням LPUSH, LRANGE, LPOP

```
127.0.0.1:6379> Sadd tech:set "Redis"
(integer) 1
127.0.0.1:6379> sadd tech:set "PostgreSQL"
(integer) 1
127.0.0.1:6379> sadd tech:set "MongoDB"
(integer) 1
127.0.0.1:6379> Smembers tech:set
1) "Redis"
2) "PostgreSQL"
3) "MongoDB"
127.0.0.1:6379> sismember tech:set "Redis"
(integer) 1
127.0.0.1:6379> |
```

Рис. 3.Виконання базових операцій Множин SADD, SMEMBERS, SISMEMBER

```
127.0.0.1:6379> HSET profile name "Oleksandr"
(integer) 1
127.0.0.1:6379> HSET profile city "Kyiv"
(integer) 1
127.0.0.1:6379> HGET profile name
"Oleksandr"
127.0.0.1:6379> HGETALL profile
1) "name"
2) "Oleksandr"
3) "city"
4) "Kyiv"
127.0.0.1:6379> |
```

Рис. 4.Використання команд Хешу HSET, HGET , HGETALL

```
127.0.0.1:6379> Zadd scores 85 "student1"
(integer) 1
127.0.0.1:6379> Zadd scores 92 "student2"
(integer) 1
127.0.0.1:6379> Zadd scores 74 "student3"
(integer) 1
127.0.0.1:6379> zrevrange scores 0 -1 WITHSCORES
1) "student2"
2) "92"
3) "student1"
4) "85"
5) "student3"
6) "74"
127.0.0.1:6379> ZRANK scores "student1"
(integer) 1
127.0.0.1:6379> |
```

Рис. 5. Використання команд ZREVRANGE - - WITHSCORES, та ZRANK для створення відсортованої множини.

```
127.0.0.1:6379> SET temp:data "Hello" EX 10
OK
127.0.0.1:6379> get temp:data
(nil)
127.0.0.1:6379> ttl temp:data
(integer) -2
127.0.0.1:6379> SET temp:data "Hello" EX 10
OK
127.0.0.1:6379> ttl temp:data
(integer) 8
127.0.0.1:6379> get temp:data
"Hello"
127.0.0.1:6379> |
```

Рис. 6. Задання часу життя об'єкту за допомогою SET - - - EX та знаходження часу життя за допомогою TTL

Аналіз особливостей роботи з типами даних у Redis

1. Рядки (Strings)

- Особливості: Це найпростіший і базовий тип даних у Redis. Рядок може зберігати будь-які дані: текст, серіалізовані JSON-об'єкти, числа або навіть бінарні файли (зображення до 512 МБ).
- Переваги та використання: Операції читання/запису відбуваються миттєво (складність $O(1)$). Оскільки команди на кшталт INCR або DECR є атомарними, цей тип ідеально підходить для реалізації лічильників (наприклад, кількості переглядів сторінки) без ризику конфліктів при одночасному доступі. Також це стандартний вибір для кешування HTML-сторінок чи токенів авторизації.

2. Списки (Lists)

- Особливості: У Redis списки реалізовані як двозв'язні списки (linked lists), а не масиви. Це означає, що додавання елементів на початок (LPUSH) або в кінець (RPUSH) відбувається з постійною швидкістю ($O(1)$) незалежно від розміру списку, але пошук елемента посередині є повільнішим ($O(N)$).
- Переваги та використання: Завдяки такій структурі списки є ідеальним інструментом для створення черг (queues), стеків (stacks), стрічок новин (наприклад, 10 останніх твітів) або систем логування подій.

3. Множини (Sets)

- Особливості: Це невпорядковані колекції унікальних рядків. Redis використовує хеш-таблиці для їх зберігання, тому додавання, видалення та перевірка наявності елемента (SISMEMBER) виконуються миттєво ($O(1)$).
- Переваги та використання: Множини незамінні, коли потрібно гарантувати унікальність даних. Їх використовують для зберігання унікальних IP-адрес відвідувачів, списку тегів до статті або друзів користувача. Крім того, Redis підтримує швидкі математичні операції над множинами: перетин (спільні друзі), об'єднання та різниця.

4. Хеші (Hashes)

- Особливості: Це асоціативні масиви (словники), які зберігають пари "поле-значення". Вони оптимізовані для зберігання об'єктів.
- Переваги та використання: Замість того, щоб зберігати об'єкт (наприклад, профіль користувача) як один великий JSON-рядок, хеші дозволяють читувати або оновлювати лише окремі поля (наприклад, змінити тільки city за допомогою HSET), не завантажуючи весь об'єкт у пам'ять. Це значно економить ресурси мережі та процесора.

5. Відсортовані множини (Sorted Sets)

- Особливості: Схожі на звичайні множини (усі елементи унікальні), але кожен елемент має додаткове числове значення — "вагу" (score). Завдяки цій вазі Redis автоматично тримає елементи відсортованими.
- Переваги та використання: Це один із найпотужніших типів даних у Redis. Він ідеально підходить для створення ігрових рейтингів (leaderboards), систем пріоритетних черг, автодоповнення пошуку або часових шкал (де "вагою" виступає час у форматі timestamp).

```
Lab_7 > redis_test.py > ...
1  import redis
2
3  # Підключення до локального сервера Redis
4  client = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6  # Збільшення лічильника
7  client.incr('script_counter')
8  counter_val = client.get('script_counter')
9  print(f"Поточне значення лічильника: {counter_val}")
10
11 # Робота зі списком задач
12 client.lpush('script_tasks', 'Write report')
13 tasks = client.lrange('script_tasks', 0, -1)
14 print(f"Список задач: {tasks}")
15
16 # Публікація повідомлення (Pub/Sub)
17 client.publish('updates', 'Lab 7 is almost done!')
18 print("Повідомлення опубліковано у канал 'updates'")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\igar1\Labs\Labs_DataBase> & C:\Users\igar1\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/igar1/Labs/Labs_DataBase/Lab_7/redis_test.py
Поточне значення лічильника: 1
Список задач: ['Write report']
Повідомлення опубліковано у канал 'updates'
PS C:\Users\igar1\Labs\Labs_DataBase> 
```

Рис. 7. Міні програма на пайтон та її виконання

Відповіді на запитання для самоперевірки

1. Як встановити Redis у Windows/Linux? У Linux встановлення відбувається через пакетний менеджер (наприклад, `sudo apt install redis-server`). На Windows найкращим способом є використання WSL (аналогічно Linux) або використання Docker контейнера командою `docker run redis`.
2. Які основні команди для роботи з рядками в Redis? SET (встановлення значення), GET (отримання значення), INCR (збільшення числового значення на 1), DECR (зменшення на 1), DEL (видалення ключа).
3. Як працювати з списками, множинами, хешами та Sorted Set? Для списків використовуються LPUSH/RPUSH та LPOP/RPOP. Для множин - SADD та SMEMBERS. Для хешів - HSET та HGET. Для Sorted Set - ZADD (з вказанням ваги/score) та ZRANGE.
4. Як встановити час життя ключа в Redis? Використовуючи параметр EX (у секундах) або PX (у мілісекундах) під час команди SET (наприклад, `SET key value EX 10`), або окремою командою EXPIRE key seconds.
5. Які можливі сценарії використання Redis у реальних застосунках? Redis ідеально підходить для кешування результатів запитів до основної БД, збереження сесій користувачів, реалізації лічильників (перегляди, лайки), створення систем обміну повідомленнями (Pub/Sub) та реалізації черг задач (Task Queues).

Висновок :

Redis є надзвичайно швидкою In-Memory структурою даних, яка оптимально підходить для завдань, що вимагають мінімальної затримки. Завдяки різноманіттю типів даних (від простих рядків до відсортованих множин) Redis дозволяє гнучко вирішувати завдання кешування, збереження сесій та аналітики в реальному часі. Наявність вбудованого механізму TTL робить його ідеальним інструментом для роботи з тимчасовими даними. Розроблений Python-скрипт підтвердив простоту інтеграції Redis у програмні застосунки.

